BANK SYSTEM

Graduation Project Documentation

Under Supervision of

Eng: Ashraf Sadeq

TEAM MEMBERS

The following talented engineers collaborated to design, build, and deliver the Bank System project under the DEPI initiative.

- Yousef Khaled
- Hossam Adel
- Amira Hassan
- Mohamed Nasser
- Abdelgalil Hekal
- Mohamed Samir

TABLE OF CONTENTS

TABLE OF CONTENTS

PROJECT OVERVIEW

1. Project Description

The Bank System is a comprehensive, enterprise-grade backend application developed as part of the Digital Egypt Pioneers Initiative (DEPI) graduation project. It provides a secure, scalable RESTful API that enables three distinct user roles — Administrators, Tellers, and Customers — to perform a wide range of banking operations through a unified, well-structured interface.

Built on ASP.NET Core with Entity Framework Core and SQL Server, the system implements modern software engineering principles including layered architecture, the repository pattern, JWT-based authentication, and role-based access control (RBAC). The API is fully documented via Swagger/OpenAPI, making it ready for integration with any frontend framework or mobile application.

2. Problem Statement

> *Traditional banking software is often monolithic, difficult to maintain, and lacks the flexibility required by modern digital services. Many systems suffer from poor security practices, insufficient role separation, and limited scalability — making them inadequate for the demands of a growing digital economy.*

Specifically, the following challenges motivated the development of this system:

- Absence of fine-grained role-based access control, causing data leaks across departments.
- Lack of automated transaction management (deposits, withdrawals, transfers) with proper audit trails.
- No standardized API contract, making frontend integration difficult and error-prone.
- Inadequate loan management and account lifecycle management features.

3. Proposed Solution

The Bank System addresses these challenges through a clean, modular backend architecture:

- Three dedicated roles (Admin, Teller, Customer) with strict endpoint-level authorization.
- Complete account lifecycle management including creation, status control, and card generation.
- Teller-operated transaction processing with full audit trail (Deposit, Withdraw, Transfer).
- Admin dashboard capabilities: customer management, teller management, branch CRUD, and aggregate statistics.
- Customer self-service: profile management, account queries, transaction history, and loan applications.
- JWT-based stateless authentication with rate limiting on sensitive endpoints.
- Swagger UI for transparent, interactive API documentation and testing.

CHAPTER 1
SYSTEM ANALYSIS

1.1 Stakeholders

Stakeholder	Role & Interest
Bank Administrator	Manages the entire platform — customers, tellers, branches, accounts, and views aggregate statistics.
Bank Teller	Processes daily transactions (deposits, withdrawals, transfers) and manages customer loans.
Bank Customer	End user who accesses their accounts, views balances, performs transfers, and applies for loans.
IT / DevOps Team	Deploys, monitors, and maintains the backend API and SQL Server database.
DEPI Initiative	Academic supervisors evaluating the graduation project quality and completeness.

1.2 Target Users

The system is designed to serve three primary user groups operating within a banking environment:

- Administrators — Back-office staff with full platform control and reporting access.
- Tellers — Front-counter staff executing financial transactions on behalf of customers.
- Customers — Retail banking clients accessing their personal financial services.

1.3 Functional Requirements

ID	REQUIREMENT	PRIORITY
FR-01	The system shall authenticate all users via JWT tokens with role-based claims.	High
FR-02	Admin shall be able to create, update, delete, and list customers.	High
FR-03	Admin shall be able to create, update, delete, and list tellers.	High
FR-04	Admin shall be able to perform full CRUD operations on bank branches.	High
FR-05	Admin shall view aggregate statistics: total deposits, withdrawals, loans, balance, fees.	High
FR-06	Teller shall perform deposit operations to customer accounts.	High
FR-07	Teller shall perform withdrawal operations from customer accounts.	High
FR-08	Teller shall perform fund transfers between accounts.	High
FR-09	Teller shall manage customer loans (create, update status).	Medium
FR-10	Customer shall view and update their own profile.	High
FR-11	Customer shall view their accounts and account balances.	High
FR-12	Customer shall view their transaction history.	High
FR-13	Customer shall create new bank accounts.	Medium
FR-14	Customer shall change their account password.	Medium
FR-15	The system shall support file upload for customer profile images.	Low

1.4 Non-Functional Requirements

ID	REQUIREMENT	PRIORITY
NFR-01	All endpoints must be secured with JWT Bearer authentication.	High
NFR-02	The login endpoint must implement rate limiting to prevent brute-force attacks.	High
NFR-03	The system must use HTTPS in production for all communication.	High
NFR-04	API response times must be under 500ms for standard CRUD operations.	Medium
NFR-05	The system must support horizontal scaling via stateless JWT authentication.	Medium
NFR-06	Global exception middleware must catch and format all unhandled errors consistently.	High
NFR-07	The database must use EF Core migrations for reproducible schema management.	Medium
NFR-08	All API contracts must be documented via Swagger/OpenAPI specification.	High
NFR-09	Password storage must use ASP.NET Identity's built-in bcrypt hashing.	High
NFR-10	The system must support paginated results for all list endpoints.	Medium

1.5 System Scope

> *The Bank System covers the core operational backend of a retail bank. It is scoped to include user identity management, account management, financial transaction processing, loan management, branch management, and reporting statistics. Out of scope are: payment gateway integrations, mobile push notifications, and front-end UI components.*

1.6 Use Cases Summary

Use Case	Actor	Description
UC-01	Admin	Manage Customers: add, view, edit, delete, search, filter by status
UC-02	Admin	Manage Tellers: full CRUD with search and count
UC-03	Admin	Manage Branches: full CRUD on bank branches
UC-04	Admin	View Statistics: deposits, withdrawals, loans, fees, balance, accounts count
UC-05	Teller	Process Deposit: credit a customer account by account type
UC-06	Teller	Process Withdrawal: debit a customer account by account type
UC-07	Teller	Process Transfer: move funds between two accounts
UC-08	Teller	Manage Loans: create loan, approve, reject, view loans
UC-09	Customer	View Profile: see personal information and account details
UC-10	Customer	Manage Accounts: view accounts, balances, transaction history
UC-11	Any User	Authenticate: login with email/password and receive JWT token

1.7 Entity Relationship Diagram (ERD)

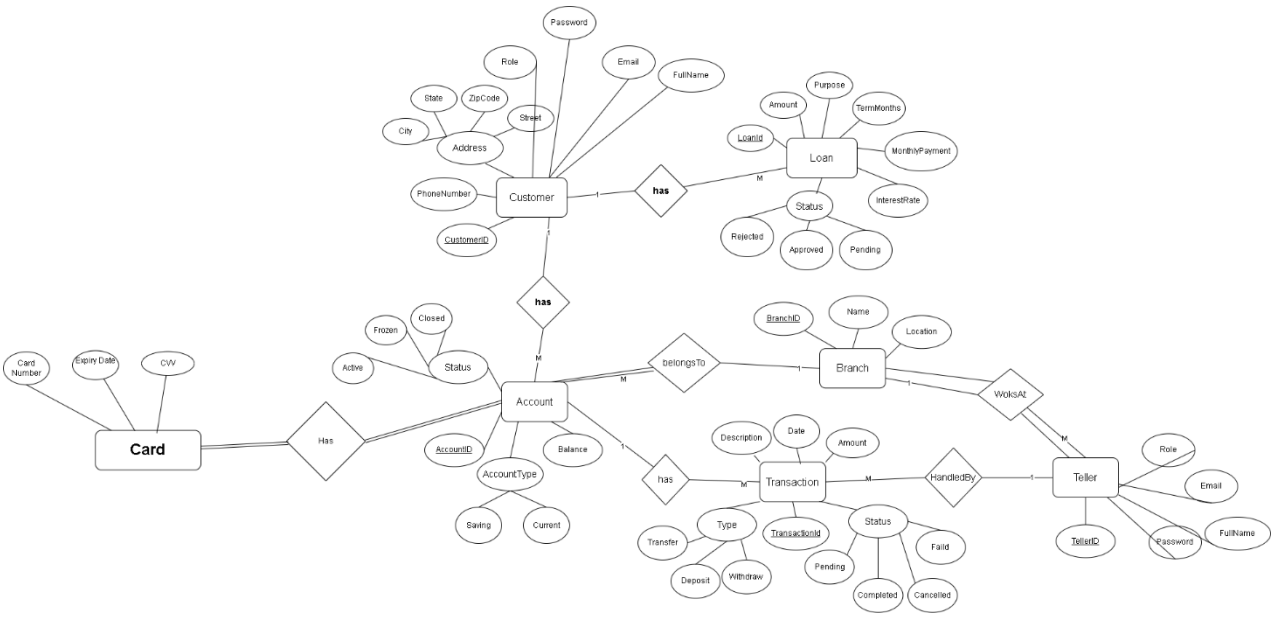


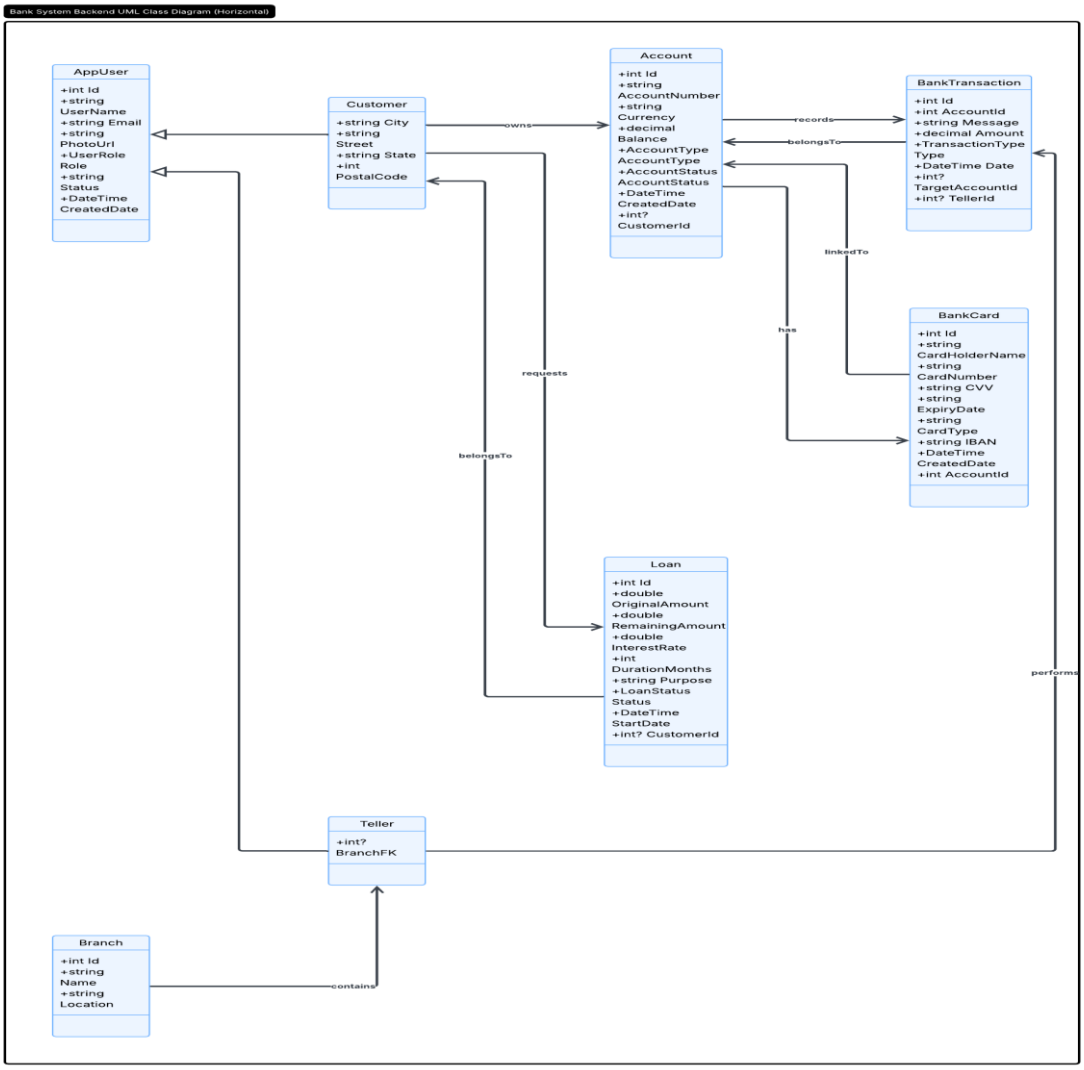
Figure 1.1 — Entity Relationship Diagram

1.8 Data Model Overview

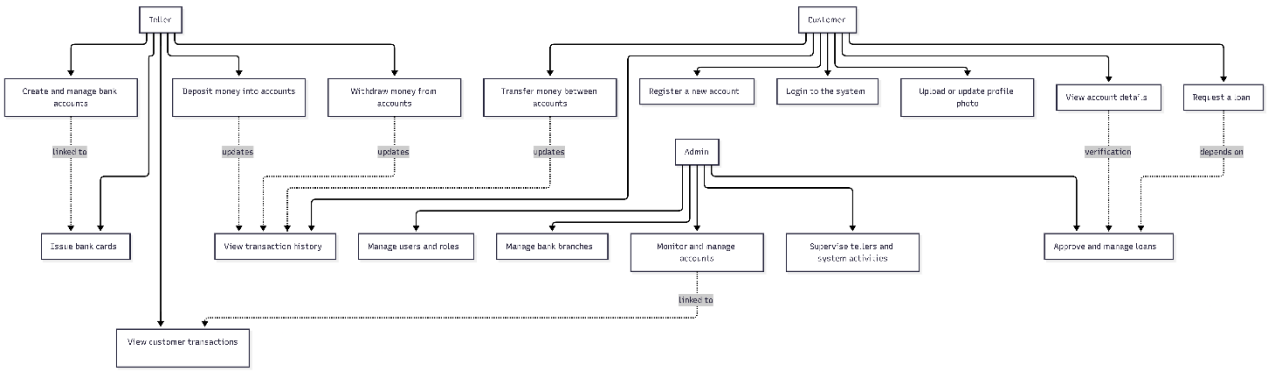
The data model is composed of the following core entities:

- AppUser — Base identity class (extends ASP.NET Identity's IdentityUser) holding UserName, Email, PhotoUrl, and role claims.
- Customer — Extends AppUser with address fields (City, Street, State, PostalCode). Has collections of Accounts and Loans.
- Teller — Extends AppUser; bank staff member assigned to process transactions.
- Account — Holds AccountNumber, Currency, Balance, AccountType (Saving/Current), AccountStatus, and belongs to a Customer.
- BankCard — One-to-one with Account; holds CardHolderName, CardNumber, CVV, ExpiryDate, CardType (Visa/MasterCard), and IBAN.
- Loan — Belongs to a Customer; tracks OriginalAmount, RemainingAmount, InterestRate, DurationMonths, Purpose, LoanStatus, and StartDate.
- Transactions — Records each financial movement: type, amount, date, source/target account reference.
- Branch — Represents a bank branch with location and contact information.

1.9 Class Diagram



1.10 Usecase Diagram



CHAPTER 2
SYSTEM ARCHITECTURE & DESIGN PATTERNS

2.1 Architecture Overview

The Bank System follows a classic N-Tier Layered Architecture, cleanly separating concerns across four distinct layers. This approach promotes maintainability, testability, and scalability by ensuring each layer has a single, well-defined responsibility.

Layer	Responsibility
Presentation	ASP.NET Core Controllers — Receive HTTP requests, validate input, return JSON responses. Decorated with [Authorize] for role enforcement.
Application	Repository interfaces (IAdminRepo, ICustomerRepo, ITellerRepo) — Define the business operations contract, decoupling logic from infrastructure.
Domain	Entity models (Account, Customer, Loan, BankCard, Transactions, Branch) — Pure data structures representing core banking concepts.
Infrastructure	EF Core DbContext, SQL Server, Repository implementations — Handle all data persistence, migrations, and query execution.

2.2 Design Patterns Used

The system employs several proven software design patterns to ensure clean, scalable, and testable code:

1. Repository Pattern

All data access logic is encapsulated behind repository interfaces. Controllers depend only on abstractions (IAdminRepo, ICustomerRepo, ITellerRepo), not on concrete implementations. This makes the system easily testable and allows swapping the data source without affecting business logic.

2. Dependency Injection (DI)

ASP.NET Core's built-in DI container wires all dependencies. Repositories are registered as scoped services, ensuring one instance per HTTP request — ideal for EF Core DbContext lifecycle management.

3. DTO Pattern

Data Transfer Objects (DTOs) are used to separate the API contract from the database schema. Dedicated DTOs (e.g., AddCustomer, EditCustomer, DisplayCustomer) prevent over-posting attacks and give full control over what data is exposed.

4. Middleware Pattern

A GlobalExceptionMiddleware intercepts all unhandled exceptions across the entire request pipeline, formats them into consistent JSON error responses, and prevents raw stack traces from leaking to clients.

5. Strategy / Role-Based Access Control

ASP.NET Core's [Authorize(Roles = "...")] attribute combined with JWT Role Claims implements strict RBAC. Each controller is locked to a specific role, preventing cross-role data access at the framework level.

2.3 Security Architecture

Security is implemented at multiple levels throughout the system:

- Authentication — JWT Bearer tokens issued on login with configurable expiry (Jwt:DurationDays in appsettings).
- Authorization — Role claims embedded in JWT; ASP.NET Core enforces role restrictions on every controller.
- Password Security — ASP.NET Identity hashes passwords using PBKDF2 with HMAC-SHA256.

- Rate Limiting — The login endpoint uses ASP.NET Core's built-in rate limiter (loginPolicy) to prevent brute-force attacks.
- Global Error Handling — Middleware prevents internal error details from being exposed to API consumers.

CHAPTER 3
API ENDPOINTS

The Bank System exposes a fully documented RESTful API organized by user role. All endpoints require a valid JWT Bearer token (except /api/Auth/login). The API follows standard HTTP conventions: GET for reads, POST for creates, PUT for updates, and DELETE for removals.

3.1 Authentication Endpoints

METHOD	ENDPOINT	DESCRIPTION
POST	/api/Auth/login	Authenticate user and receive JWT token (rate-limited)

3.2 Admin — Customer Management

METHOD	ENDPOINT	DESCRIPTION
GET	/api/Admin/customers	Get paginated list of customers (search & status filter)
GET	/api/Admin/customers/{id}	Get single customer by ID
GET	/api/Admin/customers/count	Get total customer count (with optional filters)
POST	/api/Admin/customers	Create a new customer
PUT	/api/Admin/customers/{id}	Update customer information
DELETE	/api/Admin/customers/{id}	Delete a customer
POST	/api/Admin/customers/{id}/accounts	Create a bank account for a customer

3.3 Admin — Teller Management

METHOD	ENDPOINT	DESCRIPTION
GET	/api/Admin/tellers	Get paginated list of tellers
GET	/api/Admin/tellers/{id}	Get single teller by ID
GET	/api/Admin/tellers/count	Get total teller count
POST	/api/Admin/tellers	Create a new teller
PUT	/api/Admin/tellers/{id}	Update teller information
DELETE	/api/Admin/tellers/{id}	Delete a teller

3.4 Admin — Branch Management

METHOD	ENDPOINT	DESCRIPTION
GET	/api/Admin/branches	Get all bank branches
GET	/api/Admin/branches/{id}	Get branch by ID
GET	/api/Admin/branches/count	Get total branch count

POST	/api/Admin/branches	Create a new branch
PUT	/api/Admin/branches/{id}	Update branch information
DELETE	/api/Admin/branches/{id}	Delete a branch

3.5 Admin — Statistics

METHOD	ENDPOINT	DESCRIPTION
GET	/api/Admin/stats/deposits	Get total deposit amount across all accounts
GET	/api/Admin/stats/loans	Get total loan amount outstanding
GET	/api/Admin/stats/balance	Get total balance across all accounts
GET	/api/Admin/stats/fees	Get total fees collected
GET	/api/Admin/stats/withdrawals	Get total withdrawal amount
GET	/api/Admin/stats/accounts-count	Get total number of active accounts

3.6 Teller — Transactions

METHOD	ENDPOINT	DESCRIPTION
POST	/api/Teller/deposit	Deposit amount to customer account (by account type)
POST	/api/Teller/withdraw	Withdraw amount from customer account (by account type)
POST	/api/Teller/transfer	Transfer funds between two accounts

3.7 Teller — Loan Management

METHOD	ENDPOINT	DESCRIPTION
GET	/api/Teller/loans	Get all loans with optional filters
GET	/api/Teller/loans/{id}	Get loan by ID
POST	/api/Teller/loans	Create a new loan for a customer
PUT	/api/Teller/loans/{id}/status	Update loan status (Approve / Reject / Complete)
GET	/api/Teller/loans/customer/{id}	Get all loans for a specific customer

3.8 Customer — Profile & Accounts

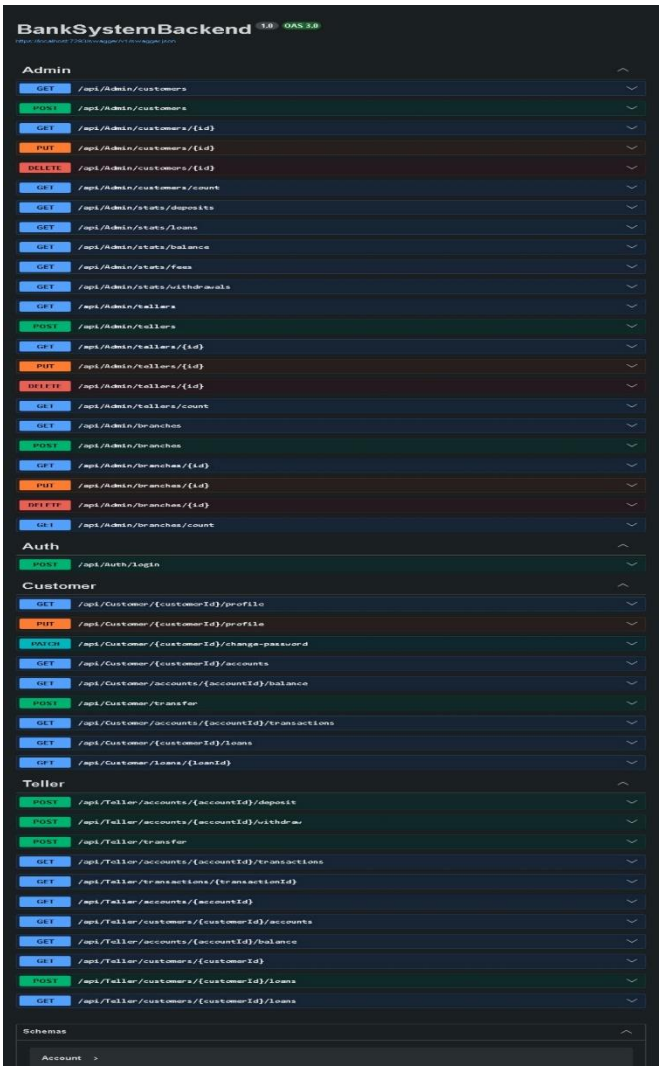
METHOD	ENDPOINT	DESCRIPTION
GET	/api/Customer/{id}/profile	Get customer profile information
PUT	/api/Customer/{id}/profile	Update customer profile
PUT	/api/Customer/{id}/change-password	Change customer password
GET	/api/Customer/{id}/accounts	Get all accounts for the customer

POST	/api/Customer/{id}/accounts	Create a new account
GET	/api/Customer/accounts/{id}/balance	Get account balance
GET	/api/Customer/accounts/{id}/transactions	Get transaction history for account
GET	/api/Customer/{id}/loans	Get all loans for the customer

3.9 Upload Endpoint

METHOD	ENDPOINT	DESCRIPTION
POST	/api/Upload/profile-image	Upload customer profile image (multipart/form-data)

API Screenshots



CHAPTER 4

TECHNOLOGIES, TOOLS & LIBRARIES

The Bank System was built using a modern, enterprise-grade technology stack. Each technology was selected for its performance, community support, and suitability for production banking applications.

TECHNOLOGY / TOOL	DESCRIPTION	ROLE IN PROJECT
ASP.NET Core 10	Microsoft's cross-platform, high-performance web framework for building APIs.	Core framework hosting all REST API controllers, middleware pipeline, DI container, and routing.
SQL Server	Microsoft's enterprise-grade relational database management system.	Primary data store for all banking entities: users, accounts, transactions, loans, branches, and cards.
Entity Framework Core	ORM (Object-Relational Mapper) for .NET — enables database access using C# objects.	Manages schema via code-first migrations; handles all CRUD queries through strongly-typed LINQ expressions.
ASP.NET Core Identity	Microsoft's membership system for authentication and user management.	Manages AppUser lifecycle, password hashing (PBKDF2), role assignment, and user claims.
JWT (JSON Web Tokens)	Industry-standard (RFC 7519) stateless authentication mechanism.	Issues signed tokens on login; each request carries role claims that ASP.NET Core enforces via [Authorize].
Swagger / OpenAPI	Industry-standard API documentation and testing framework.	Auto-generates interactive API documentation; allows developers to test all endpoints with Bearer token support.
Swashbuckle	.NET integration library for Swagger UI.	Integrates Swagger UI into the ASP.NET Core pipeline at /swagger; configured with JWT security definition.
Microsoft.AspNetCore.RateLimiting	Built-in ASP.NET Core rate limiting middleware.	Applies loginPolicy rate limiting to the authentication endpoint to prevent brute-force attacks.
AutoMapper (via DTOs)	Pattern for mapping between domain models and Data Transfer Objects.	Separates API contracts (DTOs) from domain entities, preventing over-posting and controlling data exposure.
GitHub	Cloud-based Git repository hosting platform.	Version control, collaborative development, branching strategy, and code review for the entire project.
Postman	API testing and documentation tool.	Used during development for manual API testing, creating test collections, and validating endpoint behavior.
Visual Studio 2022	Microsoft's enterprise IDE for .NET development.	Primary development environment for writing, debugging, and profiling the ASP.NET Core application.

THANK YOU

Thank you for reviewing our graduation project.

We hope this documentation reflects our dedication, teamwork, and passion for building real-world software solutions.

— Bank System Team, DEPI 2025/2026